Oracle Database Administration I

Chapter 8: Schema Objects: Tables and Constraints

محتويات الفصل:

Objectives:

1. Define schema objects and data types
2. Create and modify tables
3. Define constraints
4. View the columns and contents of a table
5. Explain the use of temporary tables

1. Naming Rules of Schema Objects:

1.1. Objects in Database:

Usually, the first objects created for an application are the tables, constraints, and indexes.

Table

A table is the basic storage container for data in a database. You create and modify the table structure via DDL statements, such as `CREATE TABLE` and `ALTER TABLE`. You access and manipulate table data via DML statements (`INSERT`, `UPDATE`, `DELETE`, `MERGE`, `SELECT`).

Tip: One important difference between DDL and DML statements is that with DML statements, you must explicitly/implicitly issue a `COMMIT` or `ROLLBACK` to end the transaction.

constraint

A constraint is a mechanism for enforcing that data adhere to business rules. Constraints inspect data as they're inserted, updated, and deleted to ensure that no business rules are violated. Almost always, when you create a table, the table needs one or more constraints defined; therefore, it makes sense to cover constraint management along with tables.

1.2. Schema in Database:

A schema is:

- Collection of database objects that are owned by a particular user
- Use SQL or Toad for Oracle to create and manipulate schema objects
- `SYS` Schema is predefined
- `SYSTEM` Schema is predefined

1.3. General Naming Rules of Schema Objects:

- The length of names must be from 1 to 30 characters
- Non quoted names cannot be Oracle-reserved words
- Non quoted names must begin with an alphabetic character from your database character set
- Names can only use letters, numbers, underscore (_), the dollar sign ($), or the hash symbol (#)
- Quoted names are not recommended

1.4. Special Naming Rules of Schema Objects:

- Database name, which may be up to 8 characters only
- Database link names, which may be up to 128 characters long

2. Understanding Table and Data Types:

2.1. Understanding Table Types:

The Oracle database supports a vast and robust variety of table types. These various types are described in the enclosed Table.

This chapter focuses on the table types that are most often used: in particular, heap organized, index organized, and temporary tables.

Table Type	Description	Typical Use
Heap Organized	The default table type and the most commonly used	Table type to use unless you have a specific reason to use a different type
Temporary	Session private data, stored for the duration of a session or transaction; space allocated in temporary segments	Program needs a temporary table structure to store and sort data; table is not required after program ends
Index organized	Data stored in a B-tree (balanced tree) index structure sorted by primary key	Table is queried mainly on primary key columns; provides fast random access
Partitioned	A logical table that consists of separate physical segments	Type used with large tables with millions of rows
External	Tables that use data stored in OS files outside the database	Type lets you efficiently access data in a file outside the database (such as a CSV file)
In-Memory External	Data that is not needed to load into Oracle storage and used for scanning as part of big data sets	Data that can be scanned for both RDBMS and Hadoop in-memory

Clustered	A group of tables that share the same data blocks	Type used to reduce I/O for tables that are often joined on the same columns
Hash clustered	A table with data that is stored and retrieved using a hash function	Reduces the I/O for tables that are mostly static (not growing after initially loaded)
Nested	A table with a column with a data type that is another table	Rarely used
Object	A table with a column with a data type that is an object type	Rarely used

2.2. Understanding Data Types:

When creating a table, you must specify the columns names and corresponding data types. As a DBA you should understand the appropriate use of each data type. Oracle supports the following groups of data types:

- Character
- Numeric
- Date/Time
- RAW
- ROWID
- LOB
- JSON

A brief description and usage recommendation are provided in the following sections.

2.2.1. Character:

Use a character data type to store characters and string data. The following character data types are available in Oracle:

VARCHAR2

The VARCHAR2 data type is what you should use in most scenarios to hold character/string data. A VARCHAR2 only allocates space based on the number of characters in the string. If you insert a one-character string into a column defined to be VARCHAR2(30), Oracle will only consume space for the one character. The following example verifies this behavior:

```
SQL> create table d(d varchar2(30));
insert into d values ('a');
select dump(d) from d;
--Here is a snippet of the output, verifying that only 1Byte has
been allocated:
DUMP(D)
----------------
Typ=1 Len=1
```

When you define a VARCHAR2 column, you must specify a length. There are two ways to do this: BYTE and CHAR. BYTE specifies the maximum length of the string in bytes, whereas CHAR specifies the maximum number of characters. For example, to specify a string that contains at the most 30B, you define it as: varchar2(30 byte)

To specify a character string that can contain at most 30 characters, you define it as: varchar2(30 char)

In almost all situations, you are safer specifying the length using CHAR. When working with multibyte character sets, if you specified the length to be VARCHAR2(30 byte), you may not get predictable results, because some characters require more than 1 byte of storage (like Arabic characters). In contrast, if you specify VARCHAR2(30 char), you can always store 30 characters in the string, regardless of whether some characters require more than 1 byte.

CHAR

A CHAR is a fixed-length character field. If you define a CHAR(30) and insert a string that consists of only one character, Oracle will allocate 30B of space. This can be an inefficient use of space. If using CHAR, it does make sense to only use it if the size of the value will not change and is absolutely static. I have normally only used CHAR if the length was under 8, and the size was absolutely fixed. The following example verifies this behavior:

```
SQL> create table d(d char(30));
insert into d values ('a');
select dump(d) from d;
--Here is a snippet of the output, verifying that 30B have been
consumed:
DUMP(D)
----------------
Typ=96 Len=30
```

NVARCHAR2 and NCHAR

The NVARCHAR2 and NCHAR data types are useful if you have a database that was originally created with a single-byte, fixed-width character set, but sometime later you need to store multibyte character set data in the same database; one of the permitted Unicode character sets. You can use the NVARCHAR2 and NCHAR data types to support this requirement.

It is simpler to standardize with use of VARCHAR2 and provide enough length to handle the multibyte characters or use the length in character instead of using NVARCHAR2 and NCHAR.

Note:

In Oracle Database 12c and higher, you can specify up to 32,767 characters in a VARCHAR2 or NVARCHAR2 data type.

To check Database Character set in database:

```
select * from nls_database_parameters where parameter like
'%CHARACTERSET';
PARAMETER                                          VALUE
```

```
NLS_NCHAR_CHARACTERSET
AL16UTF16
NLS_CHARACTERSET                                AL32UTF8
```

- `NLS_CHARACTERSET`– This is for `CHAR/VARCHAR2`. Each character takes 1 to 4 bytes to store.
- `NLS_NCHAR_CHARACTERSET` – This is for `NCHAR/NVARCHAR2`. Each character is either 2 or 4 bytes to store. So any character in `NVARCHAR2` will not occupy less than 2 bytes.

2.2.2. Numeric:

Use a numeric data type to store data that you will potentially need to use with mathematic functions, such as `SUM`, `AVG`, `MAX`, and `MIN`. Oracle supports three numeric data types:

- `NUMBER`
- `BINARY_DOUBLE`
- `BINARY_FLOAT`

For most situations, you will use the `NUMBER` data type for any type of number data. Its syntax is `NUMBER (scale, precision)` where `scale` is the total number of digits, and `precision` is the number of digits to the right of the decimal point.

What sometimes confuses DBAs is that you can create a table with columns defined as `INT`, `INTEGER`, `REAL`, `DECIMAL`, and so on. These data types are all implemented by Oracle with a `NUMBER` data type. For example, a column specified as `INTEGER` is implemented as a `NUMBER (38)`.

The `BINARY_DOUBLE` and `BINARY_FLOAT` data types are used for scientific calculations. These map to the `DOUBLE` and `FLOAT` Java data types.

Example:

With a number defined as `NUMBER (5, 2)` you can store values +/–999.99. That is a total of five digits, with two used for precision to the right of the decimal point. If defined as `NUMBER (5)` the values can be to the right or left of the decimal with a total of five digits, this value will fit, 2.4563 as would 55,555.

Note:

Oracle allows a maximum of 38 digits for a `NUMBER` data type. This is almost always sufficient for any type of numeric application. So, unless your application is performing rocket science calculations, then use the `NUMBER` data type for all your numeric requirements.

2.2.3. Date/Time:

Oracle supports three date-related data types:

Date

The `DATE` data type contains a date component as well as a time component that is granular to the second. By default, if you do not specify a time component when inserting data, then the time value defaults to midnight (0 hour at the 0 second). If you need to track time at a more granular level than the second, then use `TIMESTAMP`; otherwise, feel free to use `DATE`.

TIMESTAMP

The `TIMESTAMP` data type contains a date component and a time component that is granular to fractions of a second. When you define a `TIMESTAMP`, you can specify the fractional second precision component. For instance, if you wanted five digits of fractional precision to the right of the decimal point, you would specify that as `TIMESTAMP (5)` The maximum fractional precision is 9; the default is 6. If you specify 0 fractional precision, then you have the equivalent of the `DATE` data type.

The `TIMESTAMP` data type comes in two additional variations: `TIMESTAMP WITH TIME ZONE` and `TIMESTAMP WITH LOCAL TIME ZONE`. These are time zone–

aware data types, meaning that when the user selects the data, the time value is adjusted to the time zone of the user's session.

INTERVAL

Oracle also provides an `INTERVAL` data type. This is meant to store a duration, or interval, of time. There are two types: `INTERVAL YEAR TO MONTH` and `INTERVAL DAY TO SECOND`.

2.2.4. RAW:

The `RAW` data type allows you to store binary data in a column. This type of data is sometimes used for storing globally unique identifiers or small amounts of encrypted data. you can declare a `RAW` to have a maximum size of 32,767 bytes. The data are displayed in hexadecimal format, using characters 0–9 and A–F. When inserting data into a `RAW` column, the built-in `HEXTORAW` is implicitly applied.

2.2.5. ROWID:

`ROWID` (row identifier) is a pseudocolumn provided with every table row that contains the physical location of the row on disk.

2.2.6. LOB (Large Objects):

Oracle supports storing large amounts of data (4 GB) in a column via a LOB data type. Oracle supports the following types of LOBs:

CLOB

If you have textual data that do not fit within the confines of a `VARCHAR2`, then you should use a `CLOB` to store these data. A `CLOB` is useful for storing large amounts of character data, such as log files.

NCLOB

An `NCLOB` is similar to a `CLOB` but allows for information encoded in the nation character set of the database.

BLOB

`BLOBs` are large amounts of binary data that usually are not meant to be human readable. Typical `BLOB` data include images, audio, and video files.

BFILE

`BFILE` columns store a pointer to a file on the OS that is outside the database. When it is not feasible to store a large binary file within the database, then use a `BFILE`. `BFILEs` do not participate in database transactions and are not covered by Oracle security or backup and recovery. If you need those features, then use a `BLOB` and not a `BFILE`.

Notes :

1. CLOBs, NCLOBs, and BLOBs are known as internal LOBs. This is because they are stored inside the Oracle database. These data types reside within data files associated with the database. Whereas BFILEs are known as external LOBs.
2. The LONG and LONG RAW data types are deprecated and should not be used.

2.2.7. JSON:

Previous versions of Oracle had procedures to be able to convert table data into JSON or read JSON into the database. The JSON can be put into the database tables with JSON columns. The schema or any other details about the JSON data does not need to be known, and it can be stored in the table with other data and queried using SQL. There is definitely more to working with JSON, and there are packages to make handling the data in the database simplified. It allows for data APIs in JSON to be pulled and put into the database. Storing the JSON data in a column will allow for simple queries to be run against the database to work with other data columns.

Examples

Here is an example to create a table with a JSON column:

```
SQL> CREATE TABLE dept
(deptno NUMBER(10)
,dname VARCHAR2(14 CHAR)
,dprojects VARCHAR2(32767)
CONSTRAINT ensure_json CHECK (dprojects is JSON));
```

JSON can be inserted into the column with the other columns using an `SQL INSERT` statement, and the JSON data can be queried also using SQL:

```
SELECT dept.deptno, dept.dprojects.projectID,
dept.dprojects.projectName
from dept;
```

This will pull out of the JSON data the `projectID` and `projectName` for each of the projects contained in the data.

3. Creating a Table:

Listed next are the general factors that you should consider when creating a table:

- Type of table (heap organized, temporary, index organized, partitioned, and so on)
- Naming conventions
- Column data types and sizes
- Constraints (primary key, foreign keys, and so on)
- Index requirements
- Initial storage requirements
- Special features (virtual columns, read-only, parallel, compression, no logging, invisible columns, and so on)
- Growth requirements
- Tablespace(s) for the table and its indexes

3.1. Creating a Heap-Organized Table:

You use the `CREATE TABLE` statement(s), and data types and lengths associated with the columns. The Oracle default table type is heap organized. The term heap means that the data are not stored in a specific order in the table (instead, they're a heap of data).

Create table in SQL Plus

Here is a simple example of creating a heap-organized table:

```
SQL> CREATE TABLE dept
(deptno NUMBER(10)
,dname VARCHAR2(14 CHAR)
,loc VARCHAR2(14 CHAR));
```

If you do not specify a tablespace, then the table is created in the default permanent tablespace of the user that creates the table. Allowing the table to be created in the default permanent tablespace is fine for a few small test tables. For anything more sophisticated, you should explicitly specify the tablespace in which you want tables

created. For reference, here are the creation scripts for two sample tablespaces: HR_DATA and HR_INDEX:

```
SQL> CREATE TABLESPACE hr_data
DATAFILE '/u01/dbfile/O18C/hr_data01.dbf' SIZE 1000m
EXTENT MANAGEMENT LOCAL
UNIFORM SIZE 512k SEGMENT SPACE MANAGEMENT AUTO;
--
SQL> CREATE TABLESPACE hr_index
DATAFILE '/u01/dbfile/O18C/hr_index01.dbf' SIZE 100m
EXTENT MANAGEMENT LOCAL
UNIFORM SIZE 512k SEGMENT SPACE MANAGEMENT AUTO;
```

Usually, when you create a table, you should also specify constraints, such as the primary key. The following code shows the most common features you use when creating a table. This DDL defines primary keys, foreign keys, tablespace information, and comments:

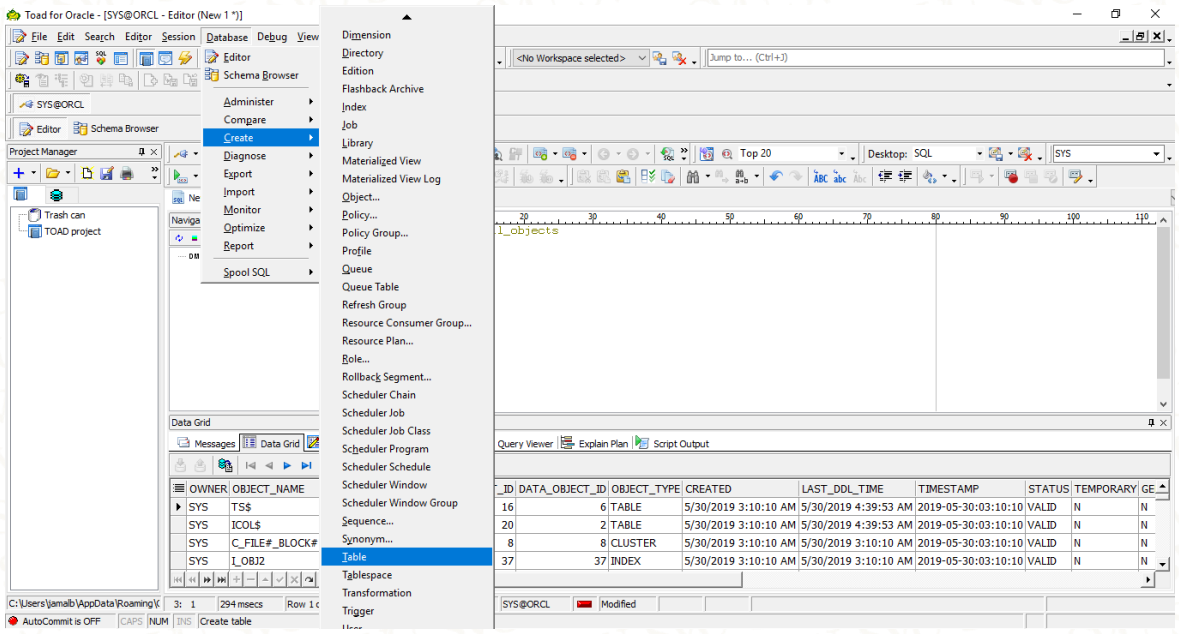
```
SQL> CREATE TABLE dept
(deptno NUMBER(10)
,dname VARCHAR2(14 CHAR)
,loc VARCHAR2(14 CHAR)
,CONSTRAINT dept_pk PRIMARY KEY (deptno)
USING INDEX TABLESPACE hr_index
) TABLESPACE hr_data;
--
SQL> COMMENT ON TABLE dept IS 'Department table';
--
SQL> CREATE UNIQUE INDEX dept_uk1 ON dept(dname)
TABLESPACE hr_index;
--
SQL> CREATE TABLE emp
(empno NUMBER(10)
,ename VARCHAR2(10 CHAR)
,job VARCHAR2(9 CHAR)
,mgr NUMBER(4)
,hiredate DATE
,sal NUMBER(7,2)
,comm NUMBER(7,2)
```

```
,deptno NUMBER(10)
,CONSTRAINT emp_pk PRIMARY KEY (empno)
USING INDEX TABLESPACE hr_index
) TABLESPACE hr_data;
--
SQL> COMMENT ON TABLE emp IS 'Employee table';
--
SQL> ALTER TABLE emp ADD CONSTRAINT emp_fk1
FOREIGN KEY (deptno)
REFERENCES dept(deptno);
--
SQL> CREATE INDEX emp_fk1 ON emp(deptno)
TABLESPACE hr_index;
```

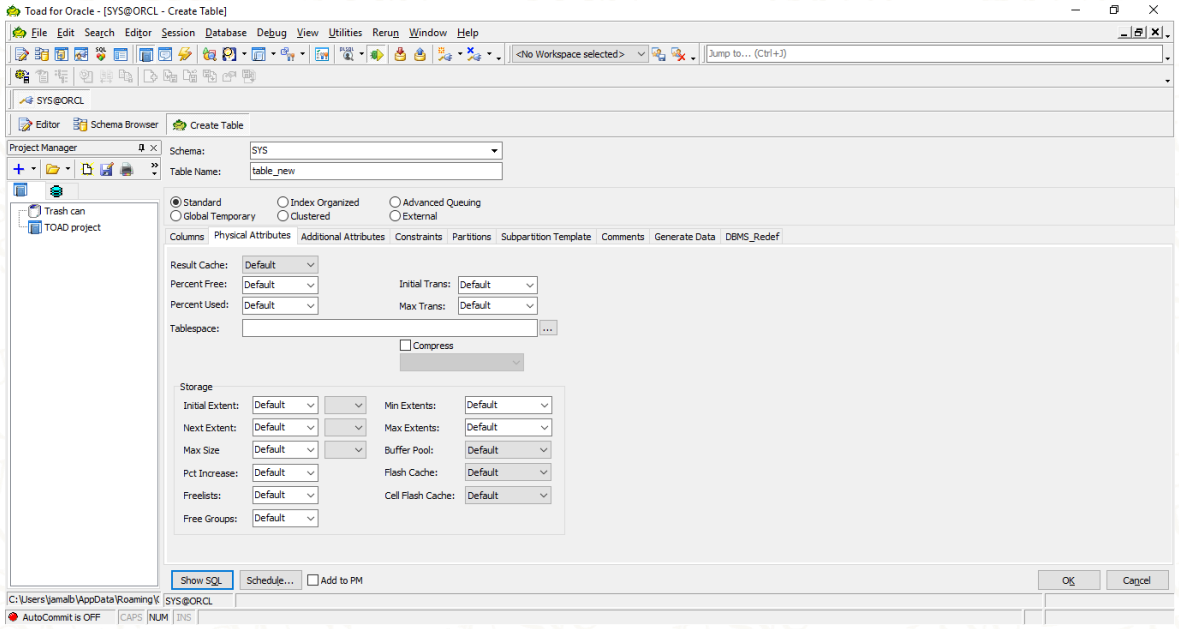
Create table in Toad (video)

If you use Toad of Oracle to create table, you will follow the following steps:

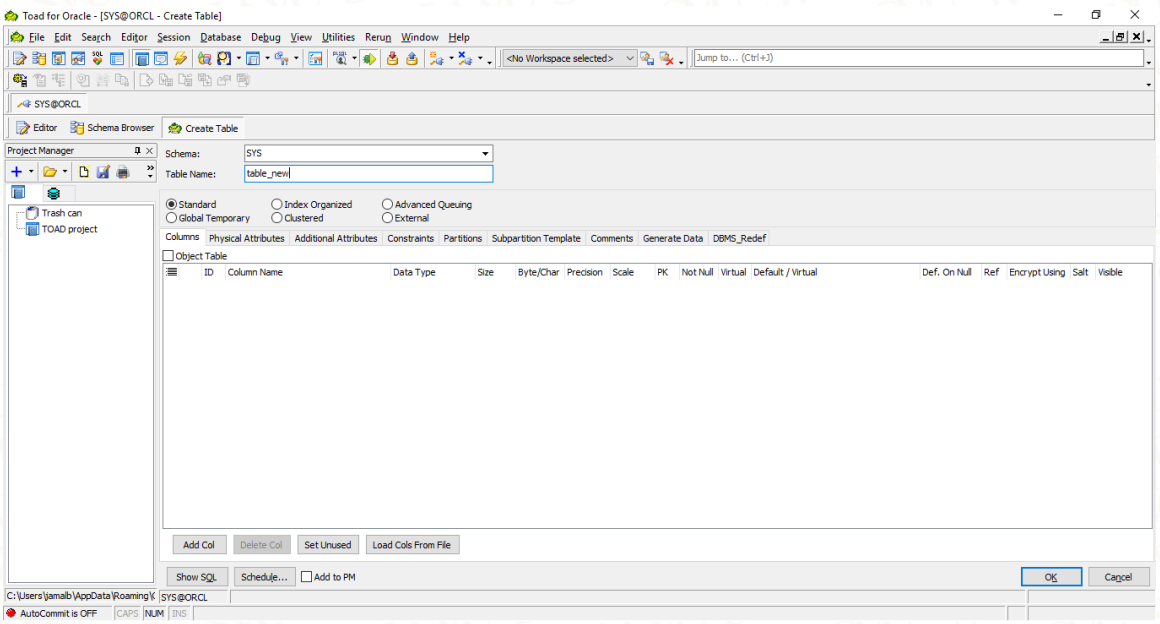
Choose create table from menu as the following:



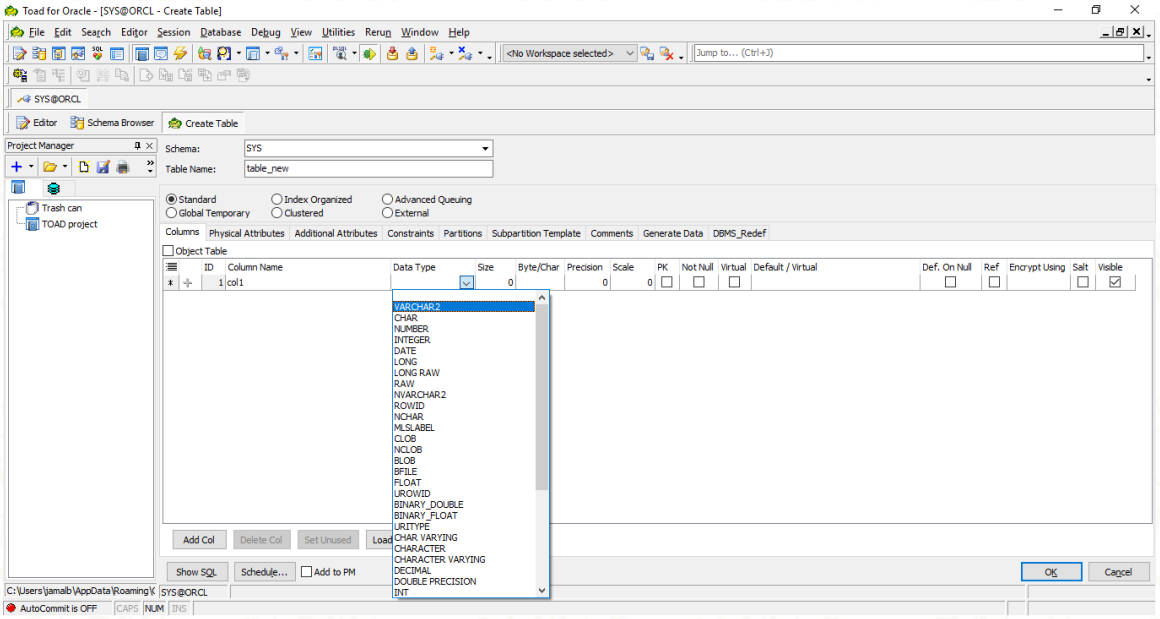
Then choose SCHEMA and TABLE NAME and TABLE TYPE and the physical attributes as the following:



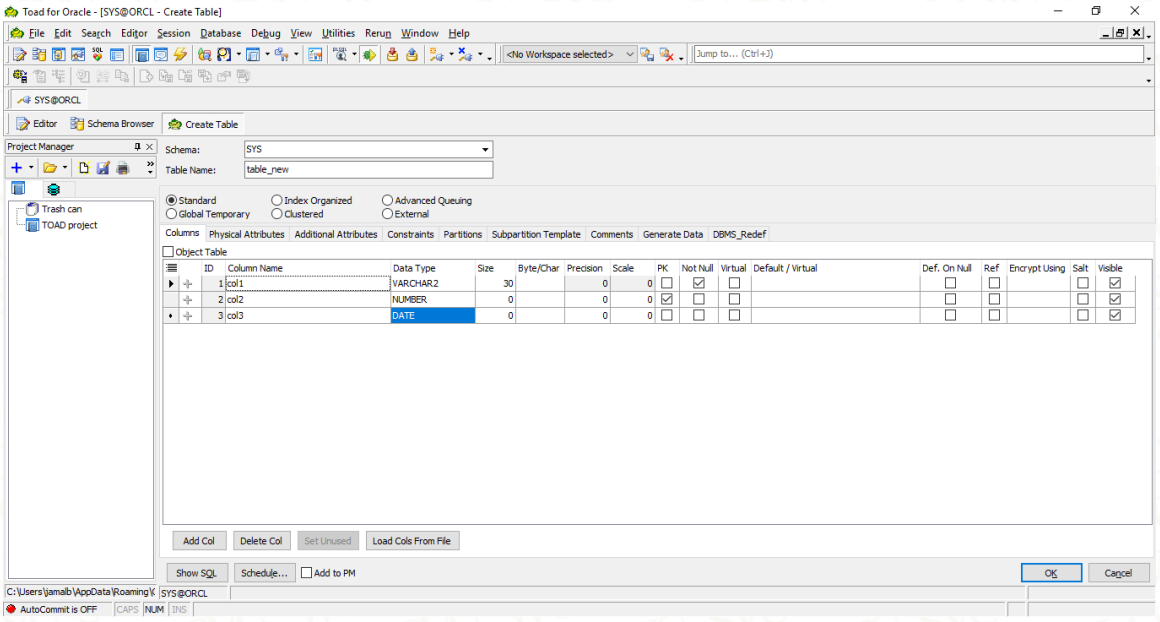
The following figure to the Additional attributes – step1:



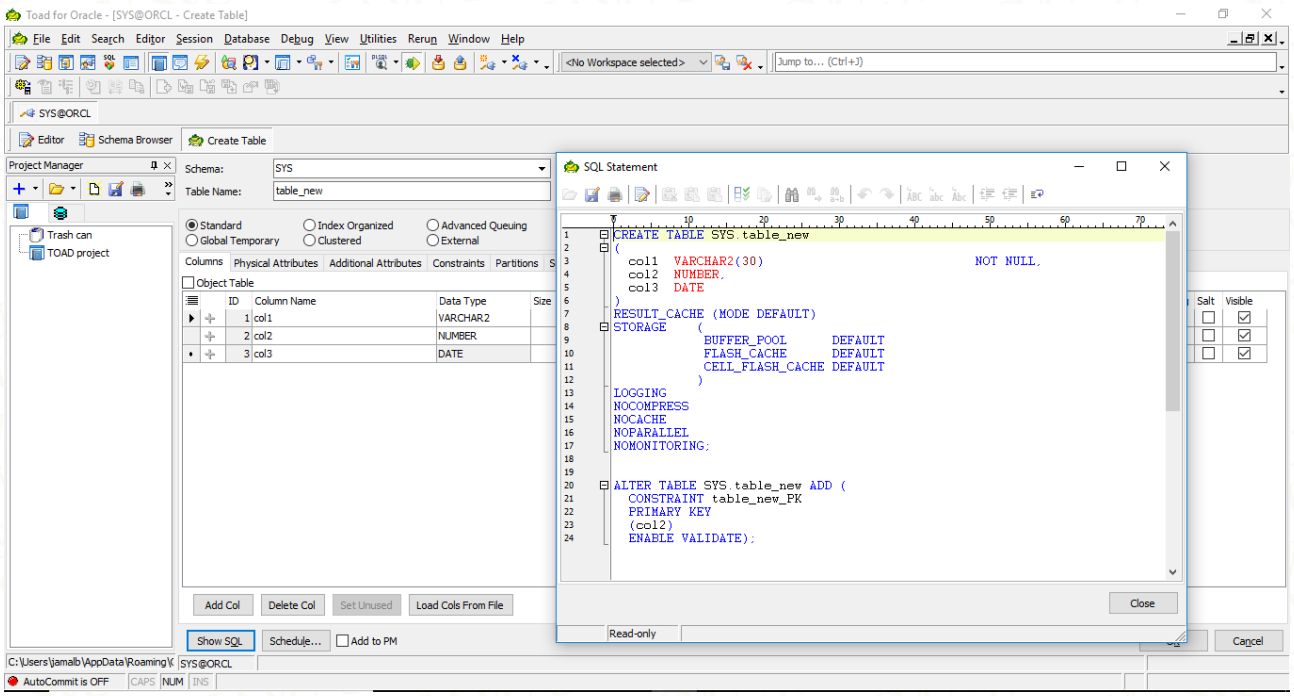
The following figure to the Additional attributes – step2:



The following figure to the Additional attributes – step3:



The following figure to see the SQL Statement of the create table as per the previous steps:



3.2. Implementing Virtual Columns:

Virtual column is based on one or more existing columns from the same table or a combination of constants, SQL functions, and user-defined PL/SQL functions, or both. Virtual columns are not stored on disk; they are evaluated at runtime, when the SQL query executes. Virtual columns can be indexed and have stored statistics.

simulate a virtual column

Prior to Oracle Database 11g, you could simulate a virtual column via a `SELECT` statement or in a view definition. For example, this next SQL `SELECT` statement generates a virtual value when the query is executed:

```
SQL> select inv_id, inv_count,
case when inv_count <= 100 then 'GETTING LOW'
when inv_count > 100 then 'OKAY' end
from inv;
```

Advantages of virtual columns

Why use a virtual column? The of doing so are as follows:

- You can create an index on a virtual column; internally, Oracle creates a function-based index.
- You can store statistics in a virtual column that can be used by the cost-based optimizer (CBO).
- Virtual columns can be referenced in `WHERE` clauses.
- Virtual columns are permanently defined in the database; there is one central definition of such a column.

example of creating a table with a virtual column

```
SQL> create table inv(inv_id number,inv_count number,inv_status
generated always as (
case when inv_count <= 100 then 'GETTING LOW'
when inv_count > 100 then 'OKAY' end) );
```

To view values generated by virtual columns, first insert some data into the table:

```
SQL> insert into inv (inv_id, inv_count) values (1,100);
```

Next, select from the table to view the generated value:

```
SQL> select * from inv;
```

Here is some sample output:

```
INV_ID     INV_COUNT       INV_STATUS
-------    --- ------      --------------
1          100         GETTING LOW
```

You can also alter a table to contain a virtual column:

```
SQL> alter table inv add(
inv_comm generated always as(inv_count * 0.1) virtual );
```

And, you can change the definition of an existing virtual column:

```
SQL> alter table inv modify inv_status generated always as(
case when inv_count <= 50 then 'NEED MORE'
when inv_count >50 and inv_count <=200 then 'GETTING LOW'
when inv_count > 200 then 'OKAY'
end);
```

suppose you want to update a permanent column based on the value in a virtual column:

```
SQL> update inv set inv_count=100 where inv_status='OKAY';
```

caveats associated with virtual columns:

- You can only define a virtual column on a regular, heap-organized table. You cannot define a virtual column on an index-organized table, an external table, a temporary table, object tables, or cluster tables.
- Virtual columns cannot reference other virtual columns.
- Virtual columns can only reference columns from the table in which the virtual column is defined.
- The output of a virtual column must be a scalar value (i.e., a single value, not a set of values).

3.3. Making Read-Only Tables:

There are several reasons why you may require the read-only feature at the table level:

- The data in the table are historical and should never be updated in normal circumstances.
- You are performing some maintenance on the table and want to ensure that it does not change while it is being updated.
- You want to drop the table, but before you do, you want to better determine if any users are attempting to update the table.

Use the `ALTER TABLE` statement to place a table in read-only mode:

```
SQL> alter table inv read only;
```

You can verify the status of a read-only table by issuing the following query:

```
SQL> select table_name, read_only from user_tables where
read_only='YES';
```

To modify a read-only table to read/write, issue the following SQL:

```
SQL> alter table inv read write;
```

4. Modifying a Table:

Altering a table is a common task. New requirements frequently mean that you need to rename, add, drop, or change column data types. When you modify a table, you must have an exclusive lock on the table. One issue is that if a DML transaction has a lock on the table, you cannot alter it. In this situation, you receive this error:

```
ORA-00054: resource busy and acquire with NOWAIT specified or
timeout expired
```

Renaming a Table

There are a couple of reasons for renaming a table:

- Make the table conform to standards
- Better determine whether the table is being used before you drop it

This example renames a table, from `INV` to `INV_OLD`:

```
SQL> rename inv to inv_old;
```

If successful, you should see this message:

```
Table renamed.
```

Adding a Column

Use the `ALTER TABLE ... ADD` statement to add a column to a table. This example adds a column to the `INV` table:

```
SQL> alter table inv add(inv_count number);
```

If successful, you should see this message:

```
Table altered.
```

Altering a Column

You need to alter a column to adjust its size or change its data type. Use the `ALTER TABLE ... MODIFY` statement to adjust the size of a column. This example changes the size of a column to 256 characters:

```
SQL> alter table inv modify inv_desc varchar2(256 char);
```

If you decrease the size of a column, first ensure that no values exist that are greater than the decreased size value:

```
SQL> select max(length(<column_name>)) from <table_name>;
```

When you change a column to `NOT NULL`, there must be a valid value for each column. First, verify that there are no `NULL` values:

```
SQL> select <column_name> from <table_name> where <column_name>
is null;
```

If any rows have a `NULL` value for the column you are modifying to `NOT NULL`, then you must first update the column to contain a value. Here is an example of modifying a column to `NOT NULL`:

```
SQL> alter table inv modify(inv_desc not null);
```

Renaming a Column

Use the `ALTER TABLE ... RENAME` statement to rename a column:

```
SQL> alter table inv rename column inv_count to inv_amt;
```

Dropping a Column

To drop a column, use the `ALTER TABLE ... DROP` statement:

```
SQL> alter table inv drop (inv_name);
```

5. Dropping a Table:

Dropping a table removes:

- Data
- Table structure
- Database triggers
- Corresponding indexes
- Associated object privileges

If you want to remove an object, such as a table, from a user, use the `DROP TABLE` statement. This example drops a table named `INV`:

```
SQL> drop table inv;
```

You should see the following confirmation:

```
Table dropped.
```

If you attempt to drop a parent table that has either a primary key or unique key referenced as a foreign key in a child table, you see an error such as:

```
ORA-02449: unique/primary keys in table referenced by foreign
keys.
```

You need to either drop the referenced foreign key constraint(s) or use the `CASCADE CONSTRAINTS` option when dropping the parent table:

```
SQL> drop table inv cascade constraints;
```

You must be the owner of the table or have the `DROP ANY TABLE` system privilege to drop a table. If you have the `DROP ANY TABLE` privilege, you can drop a table in a different schema by prepending the schema name to the table name:

```
SQL> drop table inv_mgmt.inv;
```

If you do not prepend the table name to a user name, Oracle assumes you are dropping a table in your current schema.

6. Undropping a Table:

Suppose you accidentally drop a table, and you want to restore it. First, verify that the table you want to restore is in the recycle bin:

```
SQL> show recyclebin;
```

Here is some sample output:

```
ORIGINAL     RECYCLEBIN NAME                OBJECT      DROP TIME
NAME         ---------------                TYPE        ----------
----------                                  ---------
---                                         ---
INV          BIN$0F27WtJGbXngQ4TQTwq5Hw     TABLE       2012-12-
             ==$0                                       08:12:56:45
```

Next, use the `FLASHBACK TABLE...TO BEFORE DROP` statement to recover the dropped table:

```
SQL> flashback table inv to before drop;
```

7. Removing Data from a Table:

You can use either the DELETE statement or the TRUNCATE statement to remove records from a table. You need to be aware of some important differences between these two approaches:

Features	DELETE	TRUNCATE
Choice of COMMIT or ROLLBACK	YES	NO
Generates undo	YES	NO
Resets the high-water mark to 0	NO	YES
Affected by foreign key constraints	NO	YES
Performs well with large amounts of data	NO	YES

Using DELETE

One big difference is that the DELETE statement can be either committed or rolled back. Committing a DELETE statement makes the changes permanent:

```
SQL> delete from inv;
SQL> commit;
```

If you issue a ROLLBACK statement instead of COMMIT, the table contains data as they were before the DELETE was issued.

Using TRUNCATE

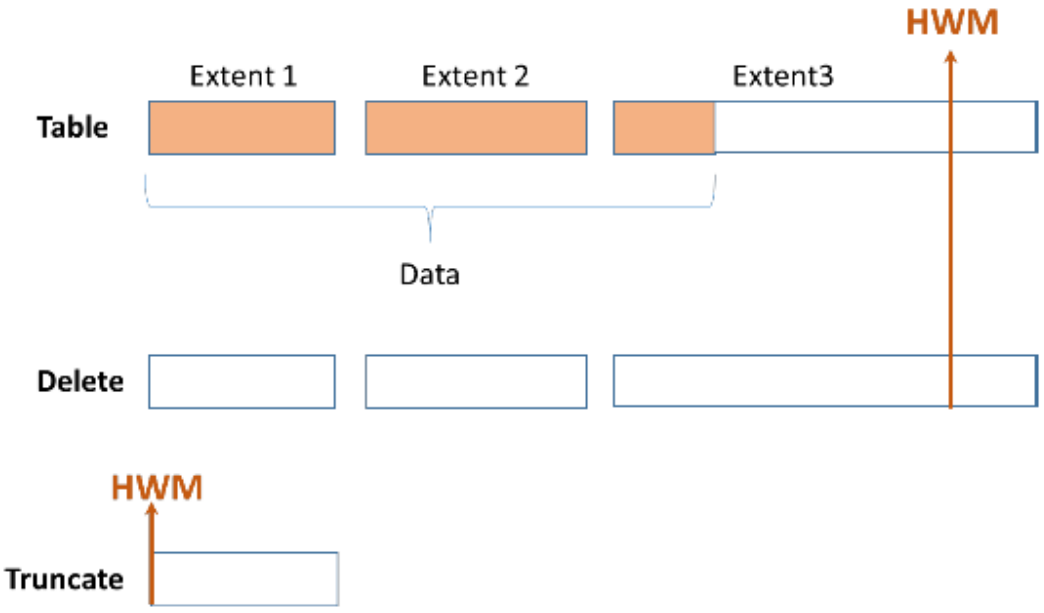
TRUNCATE is a DDL statement. This means that Oracle automatically commits the statement (and the current transaction) after it runs, so there is no way to roll back a TRUNCATE statement. If you need the option of choosing to roll back (instead of committing) when removing data, then you should use the DELETE statement. However, the DELETE statement has the disadvantage of generating a great deal of undo and redo information. Thus, for large tables, a TRUNCATE statement is usually the most efficient way to remove data:

```
SQL> truncate table computer_systems;
```

Oracle deallocates all space used for the table, except the space defined by the MINEXTENTS table-storage parameter. If you do not want the TRUNCATE statement to deallocate the extents, use the REUSE STORAGE parameter:

```
SQL> truncate table computer_systems reuse storage;
```

The TRUNCATE statement sets the high-water mark of a table back to 0. When you use a DELETE statement to remove data from a table, the high-water mark does not change. One advantage of using a TRUNCATE statement and resetting the high-water mark is that full-table scans only search for rows in blocks below the high-water mark. This can have significant performance implications.



8. Viewing and Adjusting the High-Water Mark:

Oracle defines the high-water mark of a table as the boundary between used and unused space in a segment. When you create a table, Oracle allocates a number of extents to the table, defined by the `MINEXTENTS` table-storage parameter. Each extent contains a number of blocks. Before data are inserted into the table, none of the blocks have been used, and the high-water mark is 0.

As data are inserted into a table, and extents are allocated, the high-water mark boundary is raised. A `DELETE` statement does not reset the high-water mark. You need to be aware of a couple of performance-related issues regarding the high-water mark:

- SQL query full-table scans
- Direct-path load-space usage

9. Creating a Temporary Table:

A temporary table

- Provides storage of data that is automatically cleaned up when the session or transaction ends
- Provides private storage of data for each session
- Is available for use to all sessions without affecting the private data of each session

You can create the following on temporary tables: Indexes, Views and Triggers

The following clauses control the lifetime of the rows:

- `ON COMMIT DELETE ROWS`: To specify that the lifetime of the inserted rows is for the duration of the transaction only
- `ON COMMIT PRESERVE ROWS`: To specify that the lifetime of the inserted rows is for the duration of the session

The rows will be retained until the user either explicitly deletes the data or terminates the session:

```
SQL> create global temporary table today_regs
on commit preserve rows
as select * from f_registrations
where create_dtt > sysdate - 1;
```

The records should be deleted at the end of each committed transaction:

```
create global temporary table temp_output(
temp_row varchar2(30))
on commit delete rows;
```

Note: If you do not specify a commit method for a global temporary table, then the default is `ON COMMIT DELETE ROWS`.

10. Creating an Index-Organized Table:

Index-organized tables (IOTs) are efficient objects when the table data are typically accessed through querying on the primary key. Use the `ORGANIZATION INDEX` clause to create an IOT as illustrated in the example

An IOT stores the entire contents of the table's row in a B-tree index structure. IOTs provide fast access for queries that have exact matches or range searches, or both, on the primary key. All columns specified, up to and including the column specified in the `INCLUDING` clause, are stored in the same block as the `PROD_SKU_ID` primary key column.

```
SQL> create table prod_sku
(prod_sku_id number,
sku varchar2(256),
create_dtt timestamp(5),
constraint prod_sku_pk primary key(prod_sku_id)
)
organization index
including sku
pctthreshold 30
tablespace inv_data
overflow
tablespace inv_data;
```

11. Managing Constraints:

Constraints provide a mechanism for ensuring that data conform to certain business rules. You must be aware of what types of constraints are available and when it is appropriate to use them. Oracle offers several types of constraints:

Primary key

When you implement a database, most tables you create require a primary key constraint to guarantee that every record in the table can be uniquely identified. There are multiple techniques for adding a primary key constraint to a table. The first example creates the primary key inline with the column definition:

```
SQL> create table dept(
dept_id number primary key
,dept_desc varchar2(30));
```

If you select the `CONSTRAINT_NAME` from `USER_CONSTRAINTS`, note that Oracle generates a cryptic name for the constraint (such as SYS_C003682). Use the following syntax to explicitly give a name to a primary key constraint:

```
SQL> create table dept(
dept_id number constraint dept_pk primary key using index tablespace
users,
dept_desc varchar2(30));
```

Note When you create a primary key constraint, Oracle also creates a unique index with the same name as the constraint. You can control which tablespace the unique index is placed in via the `USING INDEX TABLESPACE` clause.

The next example creates the primary key when the table is created, but not inline with the column definition:

```
SQL> create table dept(
dept_id number,
dept_desc varchar2(30),
constraint dept_pk primary key (dept_id)
using index tablespace users);
```

If the table has already been created, and you want to add a primary key constraint, use the `ALTER TABLE` statement:

```
SQL> alter table dept
add constraint dept_pk primary key (dept_id)
using index tablespace users;
```

Unique **key**

You should create unique constraints on any combinations of columns that should always be unique within a table. A unique key guarantees uniqueness on the defined column(s) within a table. you can define only one primary key per table, but there can be several unique keys. Also, a primary key does not allow a NULL value in any of its columns, whereas a unique key allows NULL values. you can use several methods to create a unique column constraint:

```
SQL> create table dept(
dept_id number
,dept_desc varchar2(30) unique);
```

If you want to explicitly name the constraint, use the CONSTRAINT keyword:

```
SQL> create table dept(
dept_id number
,dept_desc varchar2(30) constraint dept_desc_uk1 unique);
```

You can specify inline the tablespace information to be used for the associated unique index:

```
SQL> create table dept(
dept_id number
,dept_desc varchar2(30) constraint dept_desc_uk1
unique using index tablespace users);
```

You can also alter a table to include a unique constraint:

```
SQL> alter table dept
add constraint dept_desc_uk1 unique (dept_desc)
using index tablespace users;
```

And you can create an index on the columns of interest before you define a unique key constraint:

```
SQL> create index dept_desc_uk1 on dept(dept_desc) tablespace
users;
SQL> alter table dept add constraint dept_desc_uk1
unique(dept_desc);
```

Foreign key

Using a foreign key constraint is an efficient way of enforcing that data be a predefined value before an insert or update is allowed. suppose the `EMP` table is created with a `DEPT_ID` column. To ensure that each employee is assigned a valid department, you can create a foreign key constraint that enforces the rule that each `DEPT_ID` in the EMP table must exist in the `DEPT` table:

```
SQL> create table dept(
dept_id number primary key,
dept_desc varchar2(30));
```

creates a foreign key constraint on the `DEPT_ID` column in the `EMP` table:

```
SQL> create table emp(
emp_id number,
name varchar2(30),
dept_id constraint emp_dept_fk references dept(dept_id));
```

You can also specify the foreign key definition out of line from the column definition in the `CREATE TABLE` statement:

```
SQL> create table emp(
emp_id number,
name varchar2(30),
dept_id number,
constraint emp_dept_fk foreign key (dept_id) references dept(dept_id)
);
```

You can alter an existing table to add a foreign key constraint:

```
SQL> alter table emp
add constraint emp_dept_fk foreign key (dept_id)
references dept(dept_id);
```

Check

A check constraint works well for lookups when you have a short list of fairly static values, such as a column that can be either Y or N. The list of values most likely won't change, and no information needs to be stored other than Y or N, so a check constraint is the appropriate solution. If you have a long list of values that needs to be periodically updated, then a table and a foreign key constraint are a better solution. You can define a check constraint when you create a table. The following enforces the `ST_FLG` column to contain either a `0` or `1`:

```
SQL> create table emp(
emp_id number,
emp_name varchar2(30),
st_flg number(1) CHECK (st_flg in (0,1)) );
```

A slightly better method is to give the check constraint a name:

```
SQL> create table emp(
emp_id number,
emp_name varchar2(30),
st_flg number(1) constraint st_flg_chk CHECK (st_flg in (0,1))  );
```

You can also alter an existing column to include a constraint. The column must not contain any values that violate the constraint being enabled:

```
SQL> alter table emp add constraint
st_flg  check (st_flg in (0,1));
```

NOT NULL

Another common condition to check for is whether a column is null; you use the `NOT NULL` constraint to do this. The `NOT NULL` constraint can be defined in several ways. The simplest technique is shown here:

```
SQL> create table emp(
emp_id number,
emp_name varchar2(30) not null);
```

Naming the constraint will allow you to see what the constraint is for instead of a system generated constraint, which might be confused with a primary or foreign key constraint:

```
SQL> create table emp(
emp_id number,
emp_name varchar2(30) constraint emp_name_nn not null);
```

Use the `ALTER TABLE` command if you need to modify a column for an existing table. For the following command to work, there must not be any NULL values in the column being defined as `NOT NULL`:

```
SQL> alter table emp modify (emp_name not null);
```

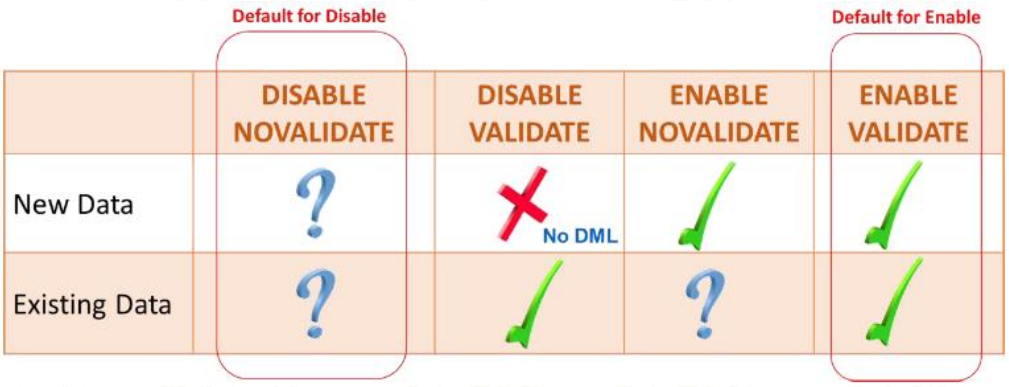
Note: If there are currently `NULL` values in a column that is being defined as `NOT NULL`, you must first update the table so that the column has a value in every row.

Constraint States

Every constraint is either enabled or disabled, and validated or not validated. Any combination of these is syntactically possible:

- `ENABLE VALIDATE`: It is not possible to enter rows that would violate the constraint, and all rows in the table conform to the constraint.
- `DISABLE NOVALIDATE`: Any data (conforming or not) can be entered, and there may already be non-conforming data in the table.

- `ENABLE NOVALIDATE`: There may already be non-conforming data in the table, but all data entered now must conform.
- `DISABLE VALIDATE`: An impossible situation: all data in the table conforms to the constraint, but new rows need not. The end result is that the table is locked against DML commands.



Use the `ALTER TABLE` statement to disable constraints on a table. In this case, there is only one foreign key to disable/enable:

```
SQL> alter table emp disable constraint emp_dept_fk;
SQL> alter table emp enable constraint emp_dept_fk;
```

You can disable a primary key and all dependent foreign key constraints with the `CASCADE` option of the `DISABLE` clause:

```
SQL> alter table dept disable constraint dept_pk cascade;
```

Note1: Keep in mind that there is no `ENABLE...CASCADE` statement.

Note2: You can get information about constraints in `dba_constraints`.

12. Quiz:

1. Which of these statements will fail because the table name is not legal? (Choose all correct answers).
   a) create table "SELECT" (col1 date);
   b) create table "lower case" (col1 date);
   c) create table number1 (col1 date);
   d) create table 1number(col1 date);
   e) create table update(col1 date);

2. Which of the following is not supported by Oracle as an internal datatype? (Choose the correct answer).
   a) CHAR
   b) FLOAT
   c) INTEGER
   d) STRING

3. Some types of constraint require an index. (Choose all that apply).
   a) CHECK
   b) NOT NULL
   c) PRIMARY KEY
   d) UNIQUE

4. Which of the following statements is incorrect about Schema object naming?
   a) Names can use letters, numbers
   b) Names can use underscore (_), the dollar sign ($)
   c) Names can use the hash symbol (#)
   d) Names can use the slash (/) and back slash (\)

5. Constraints inspect data as they're inserted, updated, and deleted to ensure that no business rules are violated.
   a) True
   b) False

6. Which of the following is not a table type?
   a) Heap Organized
   b) Temporary
   c) Partial
   d) Index organized

7. Collection of database objects that are owned by a particular user.
   a) True
   b) False

8. Which of the following character data types is not available in Oracle?
   a) `STRING`
   b) `VARCHAR2`
   c) `CHAR`
   d) `NVARCHAR2 and NCHAR`

9. When you define a VARCHAR2 column, what are two ways to specify a length. There are two ways to do this: BYTE and CHAR? (Choose the two correct ways).
   a) `BYTE`
   b) `KILO MEGA`
   c) `CHAR`
   d) `KILO GIGA`

10. What is the incorrect date/time type from the following?

a) DATE

b) TIMESTAMP

c) TIME

d) INTERVAL

11. Choose the external LOBs?

a) BLOB

b) CLOB

c) NCLOB

d) BFILE

12. Choose the internal LOBs? (Choose all the options)

a) BLOB and CLOB

b) NCLOB

c) KCLOB

d) BFILE

13. The Oracle default table type is?

a) Heap organized

b) Partitioned

c) Index organized

d) Temporary

14. If you do not specify a tablespace, then the table is created in?
    a) Default Temporary tablespace
    b) Default permanent tablespace of the user that creates the table
    c) Default permanent tablespace of the user SYS
    d) Default Temporary tablespace of the user

15. You can only define a virtual column on a regular?
    a) Index-organized table
    b) Heap-organized table
    c) External table
    d) Temporary table

16. Which of the following features is true after you use delete statement?
    a) Generates undo
    b) Resets the high-water mark to 0
    c) Affected by foreign key constraints
    d) Performs well with large amounts of data

17. Which of the following features is false after you use truncate statement?
    a) Affected by foreign key constraints
    b) Resets the high-water mark to 0
    c) Generates undo
    d) Performs well with large amounts of data

18. Virtual columns are not stored on disk; they are evaluated at runtime?
    a) True
    b) False

19. Which combination of the constraint states is syntactically impossible?

a) `ENABLE VALIDATE`

b) `DISABLE NOVALIDATE`

c) `ENABLE NOVALIDATE`

d) `DISABLE VALIDATE`

e) `DISABLE CREATION`

20. When you create a primary key constraint, Oracle also creates a unique index with the same name as the constraint?

a) True

b) False

Quiz answers – chapter 8	
1	d, e
2	d
3	c, d
4	d
5	a
6	c
7	a
8	a
9	a, c
10	c
11	d
12	a, b
13	a
14	b
15	b
16	a
17	c
18	a
19	e
20	a

References:

Books

1. Oracle Database Administrator's Guide, 18c (Oracle University), March 2019, Primary Author: Rajesh Bhatiya, Randy Urbano
2. database-new-features-guide (Oracle University), 18c E88909-01 February 2018, Primary Authors: Tanaya Bhattacharjee, Sunil Surabhi, Mark Baue
3. The Cloud DBA-Oracle: Managing Oracle Database in the Cloud - First Edition, Abhinivesh Jain and Niraj Mahajan. Apress - ISBN-13 (pbk): 978-1-4842-2634-6 ISBN-13 (electronic): 978-1-4842-2635-3

Web Sites

1. https://docs.oracle.com/database
2. https://docs.oracle.com/en/database/oracle/oracle-database/19/whats-new.html
3. http://www.oracle.com/technetwork/database/enterprise-edition/downloads/index.html